Big Data
(MEI)

João Pedro Costa
jcosta@isec.pt

1

Spark Streaming



2

Event streaming

- is the practice of capturing data in real-time from event sources
  - like databases, sensors, mobile devices, cloud services, and software applications in the form of streams of events
- storing these event streams durably for later retrieval
- manipulating, processing, and reacting to the event streams in real-time as well as retrospectively
- routing the event streams to different destination technologies as needed
- ensures a continuous flow and interpretation of data so that the right information is at the right place, at the right time



3

Event streaming

Use in a wide variety of use cases, industries and organizations

- to process payments and financial transactions in real-time
  - e.g. in stock exchanges, banks, and insurances
- to track and monitor cars, trucks, fleets, and shipments in real-time
  - e.g. as in logistics and the automotive industry
- to continuously capture and analyze sensor data from IoT devices
- to collect and immediately react to customer interactions and orders
  - e.g. as in retail, the hotel and travel industry, and mobile applications
- to monitor patients in hospital care and predict changes in condition to ensure timely treatment in emergencies
- to connect, store, and make available data produced by different business areas
- to serve as the foundation for data platforms, event-driven architectures, and micro-services

4

Streaming

- Spark Streaming
  - works on something called micro batch
    * DStream is deprecated as of Spark 3.4.0. Migrate to Structured Streaming
- Structured Streaming
  - works on the same architecture of polling the data after some duration, based on your trigger interval

5

Spark Streaming

- works on something like a micro batch
- a stream pipeline is registered with some operations
- then Spark polls the source after every batch duration (defined in the application) and then a batch is created with the received data
  - i.e. each incoming record belongs to a batch of DStream
- each batch represents an RDD



6

Spark Streaming

- extension of the core Spark API
- enables scalable, high-throughput, fault-tolerant stream processing of live data streams
- data can be ingested from many sources like TCP sockets or Kafka
- can be processed using complex algorithms expressed with high-level functions like map, reduce, join and window
- processed data can be pushed to filesystems, databases, and live dashboards
- can apply Spark's machine learning algorithms on data streams



7

Spark – basic workcount

```
[ ]: from pyspark import SparkContext
     sc = SparkContext( appName='app1')

[2]: lines =  sc.textFile("/datasets/books/shakespeare.txt")
     lines

[2]: /datasets/books/shakespeare.txt MapPartitionsRDD[1] at textFile at NativeMethodAccessorImpl.java:0

[3]: words = lines.flatMap(lambda s: s.split(" "))
     words

[3]: PythonRDD[2] at RDD at PythonRDD.scala:53

[4]: pairs = words.map(lambda s: (s,1))
     pairs

[4]: PythonRDD[3] at RDD at PythonRDD.scala:53

[5]: counts = pairs.reduceByKey(lambda a, b: a + b)
     counts

[5]: PythonRDD[8] at RDD at PythonRDD.scala:53

[6]: counts.take(5)

[6]: [('The', 4630),
      ('Project', 79),
      ('of', 17111),
      ('Shakespeare', 5),
      ('', 62184)]

[7]: sc.stop()
```


8

Spark Streaming

```
from pyspark import SparkContext
sc = SparkContext("local[2]", "NetworkWordCount")
sc.setLogLevel("ERROR")

from pyspark.streaming import StreamingContext
# Create a local StreamingContext with two working thread and batch interval of 5 second
ssc = StreamingContext(sc, 5)

# Create a DStream that will connect to hostname:port, like localhost:9999
#lines = ssc.socketTextStream("localhost", 9999)
lines = ssc.socketTextStream("10.84.128.10", 21239)

# Split each line into words
words = lines.flatMap(lambda line: line.split(" "))

# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)

# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()

ssc.start()             # Start the computation
#ssc.awaitTermination()  # Wait for the computation to terminate
ssc.awaitTerminationOrTimeout(60)
ssc.stop()
sc.stop()
```

```
-------------------------------------------
Time: 2023-06-05 17:12:00
-------------------------------------------
('Leiria', 1)
('Guarda', 1)
('Setúbal', 1)

-------------------------------------------
Time: 2023-06-05 17:12:05
-------------------------------------------
('Lisboa', 2)
('Viseu', 1)
('Guarda', 3)
('Porto', 2)
('Castelo', 2)
('Branco', 2)
('Setúbal', 1)
('Leiria', 1)
('Vila', 1)
('Real', 1)
...

-------------------------------------------
Time: 2023-06-05 17:12:10
-------------------------------------------
('Lisboa', 2)
('Viseu', 2)
('Portalegre', 2)
('Faro', 1)
('Viana', 1)
```


9

Spark Streaming

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a local StreamingContext with 2 threads and batch interval of 1s
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)

# Create a DStream that will connect to hostname:port
lines = ssc.socketTextStream("localhost", 9999)

# Split each line into words
words = lines.flatMap(lambda line: line.split(" "))

# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)

# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()

ssc.start()             # Start the computation
ssc.awaitTermination()  # Wait for the computation to terminate
```


10

Spark Streaming

- In terminal #2

```
# TERMINAL 2: RUNNING network_wordcount.py
spark-submit examples/src/main/python/streaming/network_wordcount.py
10.84.128.10 21239
```

```
-------------------------------------------
Time: 2025-05-15 15:48:51
-------------------------------------------

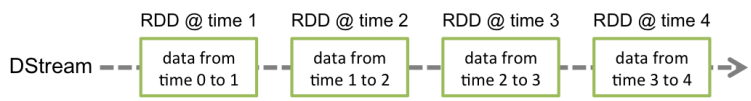
-------------------------------------------
Time: 2025-05-15 15:48:52
-------------------------------------------
('Beja', 1)
('Aveiro', 1)

-------------------------------------------
Time: 2025-05-15 15:48:53
-------------------------------------------
('Lisboa', 1)
('Porto', 1)
('Santarém', 1)
```


Spark Streaming

Discretized Stream or **DStream**

- the basic abstraction provided by Spark Streaming
- it represents a continuous stream of data, either the input data stream received from source, or the processed data stream generated by transforming the input stream.
- internally, a DStream is represented by a continuous series of RDDs, which is Spark's abstraction of an immutable, distributed dataset
- each RDD in a DStream contains data from a certain interval



Spark Streaming

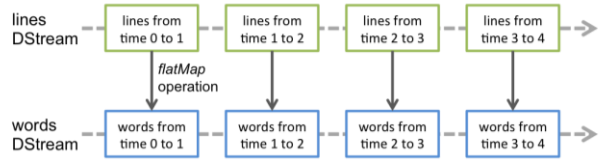
Discretized Stream or **DStream**

- operations on a DStream translates to operations on the underlying RDDs, e.g.

```
words=lines.flatMap(lambda line: line.split(" "))
```

to convert *lines* to *words*, the flatMap is applied on each RDD in the *lines* DStream to generate the RDDs of the *words* Dstream

- RDD transformations are computed by the Spark engine
- DStream operations hide details and provide developers a higher-level API



16

Spark Streaming

- input DStreams are DStreams of input data from streaming sources
- every input DStream object is associated with a Receiver object which receives the data from a source and stores it in Spark's memory for processing
- an application can receive multiple streams of data in parallel by creating multiple input DStreams
  - will create multiple receivers which simultaneously receive multiple data streams

```
lines = ssc.socketTextStream("localhost", 9999)
```

Note:

- Receiver is a long-running task (on a Spark worker/executor), hence it occupies one of the cores allocated to the Spark Streaming application

Important:

- a Spark Streaming application needs to be allocated enough cores to process the received data, as well as to run the receiver(s)

17

Spark Streaming

- Basic sources
  - sources directly available in the StreamingContext API
  - socket connections
  - file systems
- Advanced sources
  - available through extra utility classes
    e.g. sources like Kafka, Kinesis, etc

Spark Streaming

- A socket source

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a local a sparkContext
sc = SparkContext("local[2]", "NetworkWordCount")

# Create a StreamingContext
# with a batch interval of 1 second
ssc = StreamingContext(sc, 1)

# Create a DStream that will connect to hostname:port
lines = ssc.socketTextStream("localhost", 9999)
```

Spark Streaming

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a local StreamingContext with 2 threads and batch interval of 1s
sc = SparkContext("local[2]", "NetworkWordCount")
ssc = StreamingContext(sc, 1)

# Create a DStream that will connect to hostname:port
lines = ssc.socketTextStream("localhost", 9999)

# Split each line into words
words = lines.flatMap(lambda line: line.split(" "))

# Count each word in each batch
pairs = words.map(lambda word: (word, 1))
wordCounts = pairs.reduceByKey(lambda x, y: x + y)

# Print the first ten elements of each RDD generated in this DStream to the console
wordCounts.pprint()

ssc.start()             # Start the computation
ssc.awaitTermination()  # Wait for the computation to terminate
```


20

Spark Streaming

File Streams

```
streamingContext.textFileStream(dataDirectory)
```

- to read simple text files
- file streams do not require running a receiver
  - no need to allocate any cores for receiving file data
- to read data from files on any file system compatible with the HDFS API (that is, HDFS, S3, NFS, etc.), a DStream can be created as via

21

Spark Streaming

can monitor a directory `dataDir` and process files in that

- a directory can be monitored, e.g. `"hdfs://namenode:9000/input/"`
- all files in the directory will be processed as they are discovered
- the DStream will consist of all files in the directories matching the pattern, e.g.

  `hdfs://10.84.129.52:9000/examples/06-sparkstreaming/retail/*.csv`
  - it is a pattern of directories, not of files in directories

- all files must be in the same data format
- the files in the directory whose modification time is within the current window will be included in the stream
  - not its creation time

23

Spark Streaming

- once processed, changes to a file within the current window will not cause the file to be reread
  → that is: **updates are ignored**

- the more files in the directory, the longer it will take to scan for changes
  - even if no files have been modified.

- to have the file picked up in a later window, even if its contents have not changed, can use
  - `FileSystem.setTimes()` to fix the timestamp,
  - `hdfs dfs -touch` *`filename`*

24

Spark Streaming

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a local StreamingContext with 2 threads and batch interval of 5s
sc = SparkContext("local[2]", appName="PythonStreamingHDFSWordCount")

ssc = StreamingContext(sc, 5)

lines = ssc.textFileStream("/examples/06-sparkstreaming/retail/*.csv")

counts = lines.flatMap(lambda line: line.split(" "))\
              .map(lambda x: (x, 1))\
              .reduceByKey(lambda a, b: a+b)

counts.pprint()

ssc.start()
ssc.awaitTermination(60)
```


25

Spark Streaming

```
from pyspark import SparkContext
from pyspark.streaming import StreamingContext

# Create a local StreamingContext with 2 threads and batch interval of 5s
sc = SparkContext(appName="PythonStreamingHDFSWordCount")
ssc = StreamingContext(sc, 5)

# Create a local StreamingContext with 2 threads and batch interval of 1s
lines = ssc.textFileStream("hdfs://10.84.129.52:9000/examples/06-sparkstreaming/retail/*.csv")

counts = lines.flatMap(lambda line: line.split(" "))\
              .map(lambda x: (x, 1))\
              .reduceByKey(lambda a, b: a+b)

counts.pprint()

ssc.start()
ssc.awaitTermination(60)
```

```
-------------------------------------------
Time: 2023-06-05 17:35:25
-------------------------------------------

-------------------------------------------
Time: 2023-06-05 17:35:30
-------------------------------------------
('InvoiceNo,StockCode,Description,Quant
('539993,22386,JUMBO', 1)
('BAG', 44)
('PINK', 37)
('POLKADOT,10,2011-01-04', 1)
('Kingdom', 1170)
('539993,21498,RED', 1)
('RETROSPOT', 68)
('539993,22379,RECYCLING', 1)
(',5,2011-01-04', 6)
...

-------------------------------------------
Time: 2023-06-05 17:35:35
-------------------------------------------
```


26

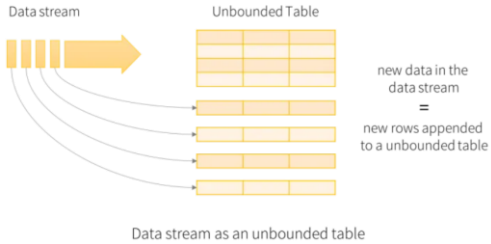
Structured Streaming

- a scalable and fault-tolerant stream processing engine
- built on the Spark SQL engine
  - can express streaming computation as batch-like query as on a static table
  - runs as an incremental query on an unbounded input table
  - will take care of running it incrementally and continuously and updating the final result as streaming data continues to arrive
  - can use dataset/dataframe API to express streaming aggregations, event-time windows, stream-to-batch joins, etc

Structured Streaming provides fast, scalable, fault-tolerant, end-to-end exactly-once stream processing without the user having to reason about streaming

Structured Streaming

- key idea - treat a live data stream as a table that is being continuously appended
- a stream processing model that is very similar to a batch processing model
- express streaming computation as standard batch-like query as on a static table
- then Spark runs it as an incremental query on the unbounded input table



Consider the input data stream as the "Input Table"
Every data item that is arriving on the stream is like a new row being appended to the Input Table

Structured Streaming

What: *is the Input Source?*

- *Streaming Input sources such as Kafka, Azure EventHub, Kinesis etc.*
- *Files systems such as HDFS, S3 etc.*
- *Sockets*

How: *to process the data, involves transformations, and sink Output mode*

- *Append - Adds new records*
- *Update - Updates changed records*
- *Complete - Overwrite all records*

When: *refers to the triggers for the streaming pipeline*

- *Processing Time - will wait for the time given before trigger*
- *Once - will trigger only one time processing all data*

Where: *the output sink where the data will be displayed/stored.*

- *Streaming Sources such as Kafka*
- *Almost all file formats*
- *Memory Sink—holds the output in memory*
- *Console Sink—displays the output in console*
- *ForEach Sink—run computation for each record.*

Structured Streaming

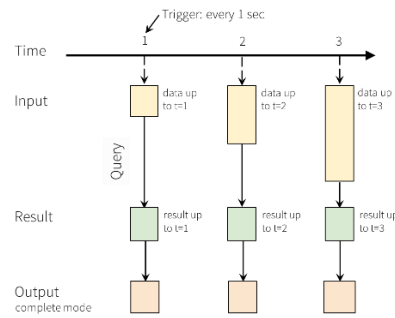
a query on the input will generate the "Result Table"

every trigger interval (e.g, `every 1 second`)

- received data in a trigger is appended to the continuously flowing data stream
- new rows get appended to the Input Table
- which eventually updates the Result Table

"Output" is defined as what gets written out to the external storage

whenever the result table gets updated, the changed result rows may be written to an external sink



Programming Model for Structured Streaming

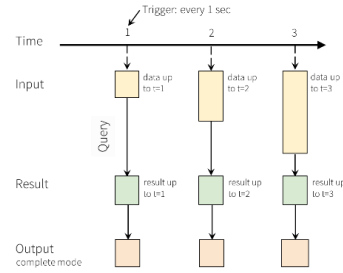
Structured Streaming

The output can be defined in different modes

- *Complete Mode* - entire updated Result Table will be written to the external storage
  - it is up to the storage connector to decide how to handle writing of the entire table
- *Append Mode* - only the new rows appended in the Result Table since the last trigger will be written to the external storage
  - only to queries where existing rows in the Result Table are not expected to change
- *Update Mode* - only the rows that were updated in the Result Table since the last trigger will be written to the external storage

Update Mode ≠ Complete Mode
- since only outputs the rows that have changed since the last trigger

Update Mode = Append Mode
- if the query doesn't contain aggregations



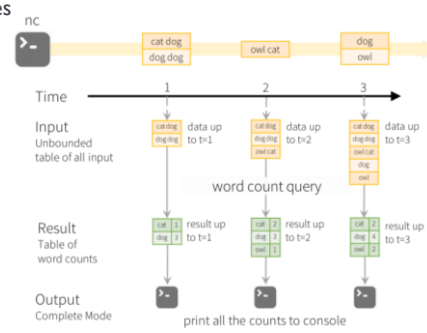
31

Structured Streaming

- works on the same architecture of polling the data after some duration, based on a trigger interval
- Internally, queries are processed using a *micro-batch processing* engine
  - processes data streams as a series of small batch jobs
  - achieving end-to-end latencies of milliseconds
  - and exactly-once fault-tolerance guarantees

Example:
Network WordCount
Source: Socket

```
# Running Netcat
$ nc -lk 9999
cat dog
cat dog
owl cat
dog
owl
```



32

Structured Streaming

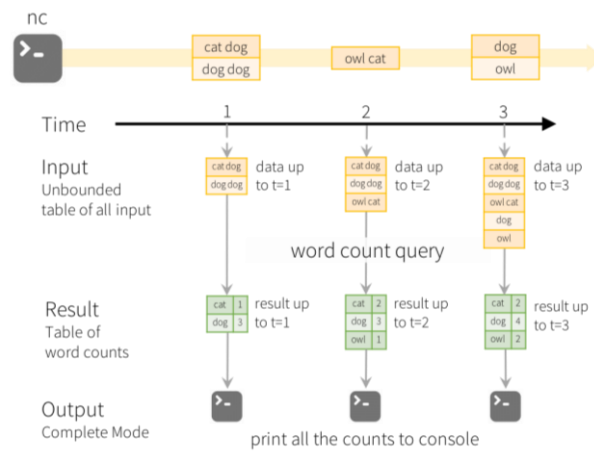
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

spark = SparkSession.builder \
    .appName("StructuredNetworkWordCount") \
    .getOrCreate()
# Create DataFrame representing the stream of input lines from connection
lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
# Split the lines into words
words = lines.select(explode(split(lines.value, " ")).alias("word"))
# Generate running word count
wordCounts = words.groupBy("word").count()

wordCounts.writeStream.format("console").outputMode("Update").start()
```


Structured Streaming

```
# Running Netcat
$ nc -lk 9999
cat dog
cat dog
owl cat
dog
owl
```



Structured Streaming



Output Sinks

- File sink - Stores the output to a directory

```
writeStream
    .format("csv")        // can be "orc", "json", "parquet", etc.
    .option("path", "path/to/destination/dir")
    .start()
```

- **Memory sink** (for debuging)

```
writeStream
    .format("memory")
    .queryName("tableName")
    .start()
```

- **Kafka sink** - Stores the output to one or more topics in Kafka

```
writeStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "10.84.128.47:9092")
    .option("topic", "sample")
    .start()
```


35

Structured Streaming




```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

spark = SparkSession.builder \
    .appName("StructuredNetworkWordCount") \
    .getOrCreate()
# Create DataFrame representing the stream of input lines from connection
lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
# Split the lines into words
words = lines.select(explode(split(lines.value, " ")).alias("word"))
# Generate running word count
wordCounts = words.groupBy("word").count()

wordCounts.writeStream.format("memory") \
     .queryName("test1").outputMode("Update").start()
```


36

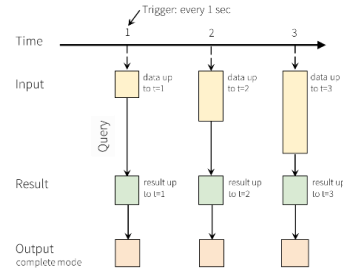
Structured Streaming

The output can be defined in different modes

- *Complete Mode* - entire updated Result Table will be written to the external storage
  - it is up to the storage connector to decide how to handle writing of the entire table
- *Append Mode* - only the new rows appended in the Result Table since the last trigger will be written to the external storage
  - only to queries where existing rows in the Result Table are not expected to change
- *Update Mode* - only the rows that were updated in the Result Table since the last trigger will be written to the external storage

Update Mode ≠ Complete Mode
- since only outputs the rows that have changed since the last trigger

Update Mode = Append Mode
- if the query doesn't contain aggregations



Structured Streaming

- can apply all kinds of operations on streaming DataFrames/Datasets
  - SQL-like operations (e.g. select, where, groupBy),
  - RDD-like operations (e.g. map, filter, flatMap)

Structured Streaming

```
# Create DataFrame representing the stream of input lines from connection
lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
# Split the lines into words
words = lines.select(explode(split(lines.value, " ")).alias("word"))

# Generate running word count
wordCounts = words.groupBy("word").count()


# or using the SQL functionalities
words.createOrReplaceTempView("table_words")

spark.sql("select count(*) from table_words")
```


Structured Streaming

Window Processing

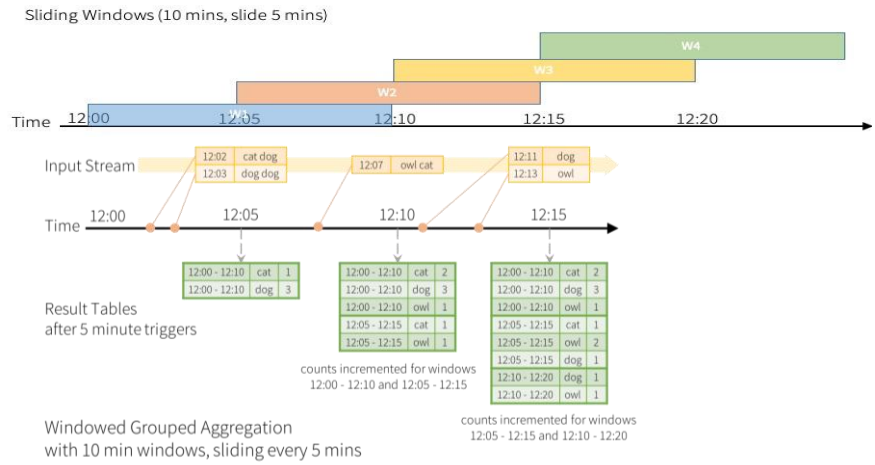
- Aggregations over a sliding event-time window are very similar to grouped aggregations
- In a grouped aggregation, aggregate values (e.g. counts) are maintained for each unique value in the user-specified grouping column.
- In case of window-based aggregations, aggregate values are maintained for each window the event-time of a row falls into

Example: count words within 10 minute windows, updating every 5 minutes



Structured Streaming

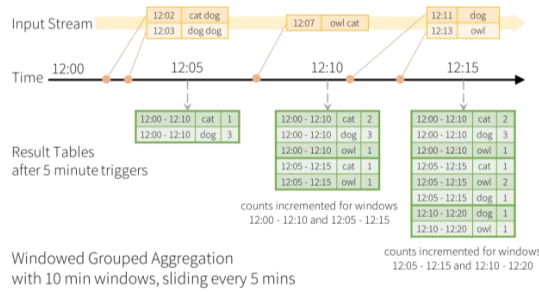
Example: count words within 10 minute windows, updating every 5 minutes



Structured Streaming

```
words = ...
# streaming DataFrame of schema { timestamp: Timestamp, word: String }

# Group the data by window and word and compute the count of each group
windowedCounts = words.groupBy(
    window( words.timestamp,
            "10 minutes",         # window size 10 minutes
            "5 minutes"),         # slide size   5 minutes
    words.word
).count()
```



Structured Streaming

Triggers

- Applied to a DataStreamWriter
- define the timing of streaming data processing, whether :
  - the query is going to be executed as micro-batch query with a fixed batch interval
  - or as a continuous processing query

```
# Default trigger (runs micro-batch as soon as it can)
df.writeStream.format("console") \
  .start()
```

```
# ProcessingTime trigger with two-seconds micro-batch interval
df.writeStream.format("console") \
  .trigger(processingTime='2 seconds') \
  .start()
```


43

Structured Streaming

Triggers

```
# Default trigger (runs micro-batch as soon as it can)
df.writeStream.format("console") \
  .start()
```

```
# ProcessingTime trigger with two-seconds micro-batch interval
df.writeStream.format("console") \
  .trigger(processingTime='2 seconds') \
  .start()
```

```
# One-time trigger (Deprecated, encouraged to use Available-now trigger)
df.writeStream.format("console") \
  .trigger(once=True) \
  .start()
```

```
# Available-now trigger
df.writeStream.format("console") \
  .trigger(availableNow=True) \
  .start()
```

```
# Continuous trigger with one-second checkpointing interval
df.writeStream.format("console") \
  .trigger(continuous='1 second') \
  .start()
```


44

Structured Streaming

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split
spark = SparkSession.builder \
    .appName("Streaming Trigger") \
    .getOrCreate()
# Create DataFrame representing the stream of input lines from connection
lines = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
# Split the lines into words
words = lines.select(explode(split(lines.value, " ")).alias("word"))
# Generate running word count
wordCounts = words.groupBy("word").count()

wordCounts.writeStream.format("console").outputMode("Update") \
          .trigger(processingTime='2 seconds') \
          .start()
```


45

Structured Streaming

DataStreamWriter foreach and foreachBatch operations

- allow you to apply arbitrary operations and writing logic on the output of a streaming query.

- foreach
  - allows custom write logic on every row,

- foreachBatch
  - allows arbitrary operations and custom logic on the output of each micro-batch
  - allows you to specify a function that is executed on the output data of every micro-batch of a streaming query

46

Structured Streaming

DataStreamWriter foreach

- allows custom write logic on every row,
- express your custom writer logic using foreach.

```
def process_row(row):
    # Write row to storage
    pass

streamingDF.writeStream.foreach(process_row).start()
```


Structured Streaming

DataStreamWriter foreach

```
def process_row(row):
    print(row, len(row.value))
    pass
```

```
dataWriter = lines.writeStream.outputMode("append").foreach(process_row)
```

```
Row(value='Leiria') 6
Row(value='Setúbal') 7
Row(value='Leiria') 6
Row(value='Faro') 4
Row(value='Viana do Castelo') 16
Row(value='Faro') 4
Row(value='Lisboa') 6
Row(value='Santarém') 8
Row(value='Vila Real') 9
Row(value='Castelo Branco') 14
```

Structured Streaming

DataStreamWriter foreach

- allows custom Class write logic on every row,
- express your custom writer logic using foreach
- express the data writing logic by dividing it into three methods:
  open, process, and close

```
class ForeachWriter:
    def open(self, partition_id, epoch_id):
        # Open connection. This method is optional in Python.
        pass

    def process(self, row):
        # Write row to connection. This method is NOT optional in Python.
        pass

    def close(self, error):
        # Close the connection. This method in optional in Python.
        pass

streamingDF.writeStream.foreach(ForeachWriter()).start()
```


Structured Streaming

DataStreamWriter foreachBatch

- allows custom write logic on every batch,

```
def foreach_batch_function(df, epoch_id):
    # Transform and write batchDF
    pass

streamingDF.writeStream.foreachBatch(foreach_batch_function).start()
```